

# Reshaping, deduplication and aggregation

Columns, rows and filters; distinct against dropDuplicates; and the aggregate functions the exam names.

Every operation in this module either changes the **columns**, the **rows**, or the **grain**. Knowing which one tells you what the output looks like.

# What each operation changes

| OPERATION                                        | COLUMNS   | ROWS                 |
|--------------------------------------------------|-----------|----------------------|
| <code>withColumn, drop, withColumnRenamed</code> | Changed   | Unchanged            |
| <code>select</code>                              | Changed   | Unchanged            |
| <code>filter, where</code>                       | Unchanged | Fewer                |
| <code>distinct, dropDuplicates</code>            | Unchanged | Fewer                |
| <code>explode</code>                             | Changed   | More, or fewer       |
| <code>groupBy with agg</code>                    | Changed   | Fewer, one per group |
| <code>join</code>                                | Changed   | More or fewer        |
| <code>pivot</code>                               | Changed   | Fewer                |

## ON THE EXAM

Note `explode`: it usually increases the row count, but it silently removes rows whose array is empty.

# Columns: adding, dropping, renaming

- ◆ `withColumn("name", expr)` adds a column, or replaces one of the same name.
- ◆ `drop("a", "b")` removes columns, and does not error if a name is absent.
- ◆ `withColumnRenamed("old", "new")` renames one column.
- ◆ `select with alias` renames several at once, and drops anything you did not list.

## ON THE EXAM

Chaining many `withColumn` calls builds a large plan. For several derived columns at once, one `select` is usually cleaner.

# Splitting a column

- ◆ **split(col, delimiter)** returns an **array**, not several columns.
- ◆ **Take a position with an index:** `split(name, ' ')[0]` gives the first element.
- ◆ **Or explode it** if each element should become its own row.
- ◆ **Indexing past the end gives null**, not an error. A missing middle name simply becomes null.

## ON THE EXAM

Splitting is a two-step idea: produce an array, then decide whether you want positions as columns or elements as rows.

# Filters

- ◆ **Multiple conditions need brackets:** `(col('a') > 5) & (col('b') == 'x')`. Python precedence bites otherwise.
- ◆ **Use `&`; and `|` and `~`,** not the Python words `and`, `or`, `not`.
- ◆ **`isin([...])` for a list,** `between(a, b)` for a range, `like` and `rlike` for patterns.
- ◆ **Filtering on a null never matches.** Use `isNull`, and remember filters silently drop null rows.

## ON THE EXAM

A filter on a column containing nulls quietly discards those rows, because a comparison with null is null, not false.

# Exploding arrays

- ◆ **explode** gives one row per element, repeating the other columns.
- ◆ **explode** drops rows whose array is empty or null. Silently.
- ◆ **explode\_outer** keeps them, with null in the exploded column.
- ◆ **posexplode** also returns the **position**, which matters when the order of elements is meaningful.

## ON THE EXAM

Explode changes the grain. After it, one row no longer means what it meant before, and any existing aggregate is now wrong.

# Pivot

- ◆ `groupBy(...).pivot("column").agg(...)` turns distinct values of a column into columns.
- ◆ **Supply the list of values** if you know it. Otherwise Spark must scan the data to find them first.
- ◆ **Without that list it is an extra job**, and a column with many distinct values produces an unusable table.
- ◆ **The reverse is stack**, turning columns back into rows.

## ON THE EXAM

Pivot is a reporting shape, so it usually belongs in gold rather than silver. Wide tables are awkward to maintain.

**distinct** compares whole rows.  
**dropDuplicates** can compare only the columns you name. That difference decides most deduplication questions.

# distinct or dropDuplicates?

`DISTINCT()`

## Whole rows

- Every column must match for two rows to be duplicates
- Cannot be told to ignore a column
- Fails to dedupe when a timestamp or file name differs
- Same as `dropDuplicates()` with no arguments

`DROPDUPPLICATES(SUBSET)`

## Named columns

- Duplicates defined by the columns you name
- Ignores differences in every other column
- Works when rows carry ingestion metadata
- Keeps one row per key — but which one?

### THE TIEBREAKER

That last question is the trap, and it gets its own slide.

dropDuplicates with a subset keeps **an arbitrary row** from each group. Not the newest, not the first. Whichever one Spark happened to see.

# Deterministic deduplication

| STEP                                            | WHAT IT DOES                                        |
|-------------------------------------------------|-----------------------------------------------------|
| <code>Window.partitionBy("order_id")</code>     | Group the rows that are candidates for each other   |
| <code>.orderBy(col("updated_at").desc())</code> | Define which one wins. Newest first, here.          |
| <code>row_number().over(window)</code>          | Number the rows within each group, 1 for the winner |
| <code>filter(col("rn") == 1)</code>             | Keep only the winner                                |

## ON THE EXAM

Four lines, and the result is reproducible. This is the pattern to reach for whenever which duplicate you keep matters.

## Two other ways to deduplicate

- ◆ **MERGE INTO on the key** deduplicates on write. A second arrival updates rather than inserting.
- ◆ **dropDuplicates on a stream needs a watermark**, or state grows forever waiting for duplicates.
- ◆ **Aggregating can deduplicate implicitly**, since grouping by the key collapses duplicates by definition.
- ◆ **Deduplicating late is expensive**. Silver is the right place, not gold.

### ON THE EXAM

MERGE is the idempotency answer from module 10 and the deduplication answer here. It is the same mechanism.

Aggregation changes the grain. One row stops meaning one transaction and starts meaning **one group**.

# The aggregate functions the exam names

| FUNCTION                                               | WHAT IT RETURNS                                      |
|--------------------------------------------------------|------------------------------------------------------|
| <code>count("*")</code>                                | Number of rows in the group                          |
| <code>count("col")</code>                              | Number of non-null values in that column             |
| <code>count_distinct("col")</code>                     | Exact number of distinct values. Expensive at scale. |
| <code>approx_count_distinct("col")</code>              | Estimated distinct count. Far cheaper.               |
| <code>mean</code> or <code>avg</code>                  | Arithmetic mean, ignoring nulls                      |
| <code>sum</code> , <code>min</code> , <code>max</code> | As expected, all ignoring nulls                      |

## ON THE EXAM

The exam guide names `count`, `approximate count distinct`, `mean` and `summary` specifically. Learn those four properly.

# approx\_count\_distinct

- ◆ **Estimates the distinct count** using a sketch, rather than tracking every value seen.
- ◆ **The rsd parameter is the maximum relative standard deviation.** Default 0.05, meaning about 5%.
- ◆ **Below rsd 0.01 it is more efficient to use count\_distinct** instead. The docs say so directly.
- ◆ **Use it when the trend matters more than the exact figure**, such as daily active users on a dashboard.

## ON THE EXAM

An estimate within 5% is usually fine for a trend line and completely unacceptable for a regulatory count. The requirement decides.

# describe and summary

- ◆ **describe()** gives count, mean, standard deviation, min and max for numeric columns.
- ◆ **summary()** gives those plus the 25th, 50th and 75th percentiles by default.
- ◆ **summary accepts arguments**, so you can request specific statistics.
- ◆ **Both are for exploration**, giving a quick profile of unfamiliar data.

## ON THE EXAM

summary is the richer of the two. Its quartiles are what reveal skew, which describe's mean and max cannot show you.

## groupBy and agg

- ◆ `groupBy("a").agg(sum("x").alias("total"), count("*").alias("n"))` is the general shape.
- ◆ **Always alias your aggregates.** Otherwise you get column names like `sum(x)`.
- ◆ **Group by several columns** by passing several names.
- ◆ **Only grouped or aggregated columns can be selected.** Anything else is ambiguous, and Spark says so.

### ON THE EXAM

Aliasing is not cosmetic. A downstream query referring to a column called `sum(x)` is fragile and awkward to write.

# Filter before or after aggregating?

## FILTER FIRST

### WHERE, in SQL terms

- Removes rows before the calculation
- The aggregate is computed on fewer rows
- Changes the value of every aggregate
- Also faster, since less data is grouped

## FILTER AFTER

### HAVING, in SQL terms

- Removes groups after the calculation
- The aggregate is computed on everything
- Changes which groups appear, not their values
- Needed when the condition is about the aggregate

## THE TIEBREAKER

'Customers whose orders over £100 total more than £1000' filters first. 'Customers whose total exceeds £1000' filters after.

# When you need the detail as well

- ◆ **groupBy collapses rows.** You get one row per group and the detail is gone.
- ◆ A window function keeps every row and adds the aggregate alongside it.
- ◆ `sum("x").over(Window.partitionBy("customer"))` gives each row its customer's total.
- ◆ **Use a window when you need row and group together**, such as each order's share of the customer's spend.

## ON THE EXAM

Same aggregate, different grain. `groupBy` answers 'what is the total per customer'; a window answers 'what is this row's share of it'.

# Tiebreakers from this module

| WHEN A QUESTION SAYS                   | REACH FOR                                                                               |
|----------------------------------------|-----------------------------------------------------------------------------------------|
| distinct did not remove the duplicates | The rows differ in some column. Use <code>dropDuplicates(subset)</code> .               |
| A different row is kept each run       | <code>dropDuplicates</code> picks arbitrarily. Use <code>row_number</code> in a window. |
| One row per array element              | <code>explode</code> , or <code>explode_outer</code> to keep empty arrays               |
| Split a column into parts              | <code>split</code> returns an array. Index it, or <code>explode</code> it.              |
| Exact distinct count is too slow       | <code>approx_count_distinct</code> , <code>rsd</code> default 0.05                      |
| Quartiles or a data profile            | <code>summary</code> , which is richer than <code>describe</code>                       |
| Groups whose total exceeds a threshold | Filter after the aggregation                                                            |
| Each row's share of its group's total  | A window function, not <code>groupBy</code>                                             |

## ON THE EXAM

Eight lines. The first two are a pair, and they are the most valuable in the module.

A bronze orders table receives the same order twice whenever an upstream retry fires. Each bronze row carries `_source_file` and `_ingested_at` columns added at load time.

An engineer adds `.distinct()` to the silver build to remove the duplicates. The duplicate orders are still there.

## Why did distinct not work?

- ☐ A distinct only works on a single column and must be given one
- ☐ B The rows are not identical: `_source_file` and `_ingested_at` differ, so distinct sees two different rows
- ☐ C distinct is lazy and needs an action before it takes effect
- ☐ D distinct only removes adjacent duplicate rows, so the data must be sorted first

### ◆ PAUSE AND DECIDE

## Why did distinct not work?

- A

 distinct needs a column argument  
*distinct takes no arguments and compares whole rows. That is precisely the problem, but this describes the API incorrectly.*
- B

 The ingestion metadata differs, so the rows are not identical  
*Correct. distinct compares every column. Two loads of the same order differ in \_source\_file and \_ingested\_at, so they are genuinely different rows.*
- C

 distinct is lazy  
*It is lazy, and that is irrelevant. The silver table was written, so an action ran and the transformation executed.*
- D

 Only removes adjacent duplicates  
*Describes the Unix uniq command, not Spark. distinct is a global operation and needs no sorting.*

### THE SIGNAL

The ingestion metadata that makes bronze traceable is exactly what makes distinct useless on it. Both things are true at once.

Following the fix, the engineer uses `dropDuplicates(['order_id'])`. The duplicates are gone. Weeks later, an analyst notices that the delivery address on some orders changes between one daily run and the next, even though nothing upstream changed.

Where an order was received twice, the two bronze rows differ: the second correction included an updated address.

### What is happening, and what is the fix?

- ☐ A `dropDuplicates` keeps an arbitrary row per key; use `row_number` over a window ordered by `_ingested_at desc`
- ☐ B `dropDuplicates` keeps the first row by file order; sort the DataFrame before calling it
- ☐ C The bronze table is being rewritten between runs
- ☐ D `dropDuplicates` should be given all columns so it picks the complete row

### ◆ PAUSE AND DECIDE

## What is happening, and what is the fix?

**A** Arbitrary row; use row\_number over a window

Correct. dropDuplicates gives no guarantee about which row survives. The window pattern states the rule explicitly and is reproducible.

**C** Bronze is being rewritten

Bronze is append-only and the scenario says nothing upstream changed. It invents a cause when a sufficient one is present.

**B** It keeps the first row; sort beforehand

*There is no first-row guarantee to rely on, and sorting before dropDuplicates does not make the choice deterministic.*

**D** Pass all columns

*That makes it equivalent to distinct, which returns to scenario 1's problem: both rows survive because they differ.*

### THE SIGNAL

Non-determinism is the symptom to recognise. The same input producing different output means something is choosing arbitrarily.

A team needs a report of customers whose total spend across the year exceeds £10,000. An engineer writes: filter total\_spend greater than 10000, then groupBy customer, then sum the amounts.

The report returns far fewer customers than the business expects, and the totals shown are lower than the customers' actual annual spend.

## What is wrong?

- ☐ A The filter is applied to individual orders before grouping, so only customers with a single order over £10,000 appear
- ☐ B sum ignores nulls, so some order amounts were excluded
- ☐ C groupBy should have been a window function
- ☐ D The filter should use >= rather than >

### ◆ PAUSE AND DECIDE

## What is wrong?

**A** The filter runs before grouping, on individual orders  
Correct. Filtering first removes rows, so only large single orders survive to be grouped, and each customer's total is computed from those alone.

**C** Should have been a window function  
*A window would keep every row, which is not what a customer-level report wants. The problem is the filter's position, not the aggregate.*

**B** sum ignores nulls  
*True of sum, and it would not produce this pattern. It also does not explain why fewer customers appear.*

**D** Use  $\geq$  instead of  $>$   
*An off-by-one on the boundary would change the result by a handful of customers, not produce systematically lower totals.*

### THE SIGNAL

Two symptoms, one cause: fewer customers AND lower totals. Filtering after grouping would change which customers appear but not their totals.

A dashboard shows daily unique visitors across 400 million events. It uses `count_distinct` on `visitor_id`, grouped by day. The query takes eleven minutes and the dashboard times out.

The dashboard is used to spot week-on-week trends. A separate regulatory report also counts unique visitors, and must be exact.

## What should the team do?

- A Use `approx_count_distinct` for both, since it is far faster
- B Use `approx_count_distinct` for the dashboard and keep `count_distinct` for the regulatory report
- C Use `approx_count_distinct` with `rsd` set to 0.001 for both, giving speed and accuracy
- D Keep `count_distinct` for both and increase the SQL warehouse size

## ◆ PAUSE AND DECIDE

## What should the team do?

- A

 Approximate for both  
*Fast, and it makes the regulatory figure an estimate. 'Must be exact' is a stated requirement, not a preference.*
- B

 Approximate for the dashboard, exact for the report  
*Correct. The requirements differ, so the answers differ. A 5% estimate is fine for a trend and unacceptable for a regulatory count.*
- C

 rsd 0.001 for both  
*The docs note that below rsd 0.01 it is more efficient to use count\_distinct. At 0.001 you pay for precision and lose the speed advantage.*
- D

 Exact for both, bigger warehouse  
*Buys time on the dashboard at permanent cost, and the exact count still scales badly as events grow.*

### THE SIGNAL

Two consumers, two requirements. Nothing says one query must serve both, and the exam rewards noticing when it does not.

Ask what one row means before the operation, and what it means after. **Every question in this module is answered by that.**